



Trailhead



[Apex & .NET Basics \(/content/learn/modules/apex_basics_dotnet?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](/content/learn/modules/apex_basics_dotnet?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)



Understand Execution Context ▾

Understand Execution Context

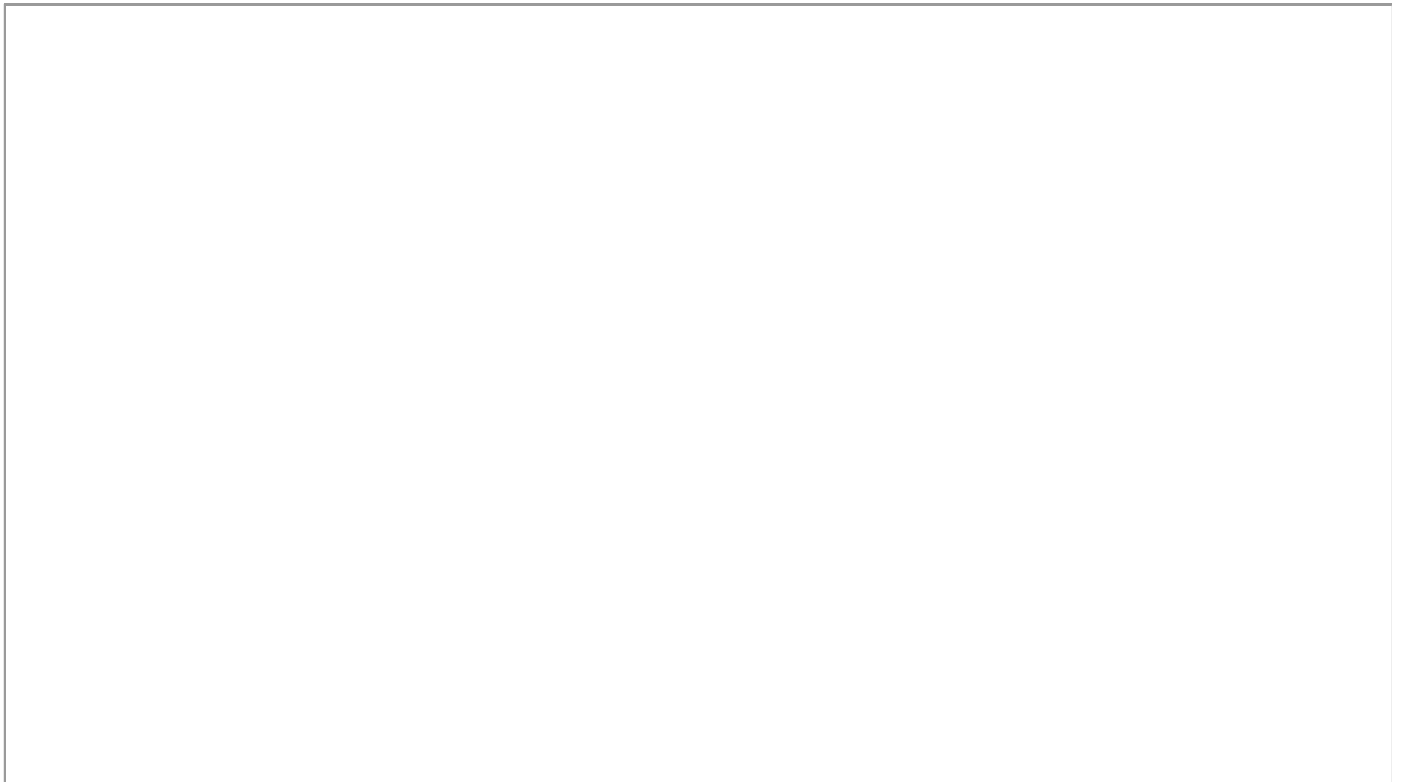
Learning Objectives

After completing this unit, you'll be able to:

- Know which methods to use to invoke Apex
- Write a trigger for a Salesforce object
- Observe how execution context works by executing code in Developer Console
- Understand how governor limits impact design patterns
- Understand the importance of working with bulk operations

Follow Along with Trail Together

Want to follow along with an expert as you work through this step? Take a look at this video, part of the Trail Together series.



(This clip starts at the 16:36 minute mark, in case you want to rewind and watch the beginning of the step again.)

What Is Execution Context?

For ASP.NET applications, code is executed in the context of an application domain. In the Lightning Platform world, code executes within an execution context. In a nutshell, this context represents the time between when the code is executed and when it ends. The important thing for you to understand is that the Apex code you write is not always the only code that is executing.





Method	Description
Database Trigger	Invoked for a specific event on a custom or standard object.
Understand Execution Context	
Anonymous Apex	Code snippets executed on the fly in Dev Console & other tools.
Asynchronous Apex	Occurs when executing a future or queueable Apex, running a batch job, or scheduling Apex to run at a specified interval.
Web Services	Code that is exposed via SOAP or REST inbound or outbound web services.
Email Services	Code that is set up to process inbound or outbound email.
Visualforce or Lightning Pages	Visualforce controllers and Lightning components can execute Apex code automatically or when a user initiates an action, such as clicking a button. Lightning components can also be executed by Lightning processes and flows.

Besides invoking Apex code, actions, such as creating a new task, sending an email, performing a field update, or sending an outbound message, can all be triggered by one of the declarative platform features. These actions also run within an execution context.

Another important consideration is the context of the user executing the Apex code. By default, Apex executes in system context. Apex code has access to all objects and fields. Object permissions, field-level security, and sharing rules aren't applied for the current user. You can use the `with sharing` keyword to specify that the sharing rules for the current user be taken into account for a class. This topic is important, so check out [Using the with sharing or without sharing keywords \(https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_keywords_sharing.htm\)](https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_keywords_sharing.htm).

Trigger Essentials

Before learning more about the execution context, let's take a step back to introduce you to the world of database triggers. Similar to triggers in SQL Server, Apex database triggers execute programming logic before or after events to records in Salesforce. When defining the trigger, you can specify more than one of the following events:

- `before insert` [copy](#)
- `before update` [copy](#)
- `before delete` [copy](#)
- `after insert` [copy](#)
- `after update` [copy](#)
- `after delete` [copy](#)
- `after undelete` [copy](#)

The basic syntax for a trigger looks like the following:

```
1 trigger TriggerName on ObjectName (trigger_events) {
2     // code_block
3 }
```





There are two types of triggers.

- **Before triggers** are used to update or validate record values before they're saved to the database. Apex & .NET Basics (/content/learn/modules/apex_basics_dotnet?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)
- **After triggers** are used to access field values that are set by the system (such as a record's `Id` or `LastModifiedDate` field), and to affect changes in other records. The records that fire the after trigger are read-only.

Understanding Execution Context



The system saves the records that fired the before trigger after the trigger finishes execution. You can modify the records in the trigger without explicitly calling a DML insert or update operation. If you perform DML statements on those records, you get an error.

Now, we know how much .NET developers love to solve problems with code, but we have a super big productivity tip for you here. **You only want to resort to using a trigger when you are absolutely sure that the same thing cannot be accomplished with one of our point-and-click automation tools.**

To make your life easier, the Salesforce platform includes a powerful automation tool, Flow Builder, for managing business logic without writing code. In most cases, tasks that could once upon a time only be accomplished with a trigger are now better suited for one of the automation tools.



If you're a developer new to the platform, definitely take the time to check out the [Approve Records with Approval Processes](https://trailhead.salesforce.com/content/learn/modules/business_process_automation) (https://trailhead.salesforce.com/content/learn/modules/business_process_automation) module in the Developer Beginner trail before creating a trigger. We're pretty sure that you don't want to be known as the guy or gal who brought a lot of unnecessary technical overhead into the Salesforce org.

Mark Execution Context

To better understand execution context, let's walk through creating an Apex database trigger that creates an opportunity when a new account is entered. This trigger calls a method from a handler class, so we first need to create that.

1. From Setup (⚙️), select **Developer Console**.
2. In the Developer Console, select **File > New > Apex Class**.
3. Enter `AccountHandler` for the class name and click OK.
4. Delete the existing code, and insert the following snippet:

```
1 public with sharing class AccountHandler {
2     public static void CreateNewOpportunity(List<Account> accts) {
3         for(Account acct : accts) {
4             Opportunity opp = new Opportunity();
5             opp.Name = acct.Name + ' Opportunity';
6             opp.AccountId = acct.Id;
7             opp.StageName = 'Prospecting';
8             opp.CloseDate = System.Today().addMonths(1);
9             insert opp;
10        }
11    }
12 }
```



5. Press **Ctrl + S** to save your class.



(https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_triggers_bestpract.htm).

Apex & .NET Basics (/content/learn/modules/apex_basics_dotnet?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

So now that we have the handler class, we create the Account trigger.

Understand Execution Context

1. In Developer Console, select **File > New > Apex Trigger**.
2. Enter `AccountTrigger`  as the name, and select **Account** as the sObject.
3. Click **Submit**.
4. Delete the existing code, and insert the following snippet:

```
1 trigger AccountTrigger on Account(before insert, before update, before
2   delete, after insert, after update, after delete, after undelete) {
3   if(Trigger.isAfter && Trigger.isInsert) {
4       AccountHandler.CreateNewOpportunity(Trigger.New);
5   }
6 }
```



5. Press **Ctrl + S** to save your trigger.

To complete the walkthrough, we execute anonymous code to simulate a user entering a new Account using the Salesforce interface. Remember, Apex code can be executed in a number of different ways.

1. In Developer Console, select **Debug > Open Execute Anonymous Window**.
2. Delete any existing code, and insert the following snippet:

```
1 Account acct = new Account(
2   Name='Test Account 2',
3   Phone='(415) 555-8989',
4   NumberOfEmployees=50,
5   BillingCity='San Francisco');
6 insert acct;
```



3. Make sure that the **Open Log** option is selected and click **Execute**.
A new tab shows the execution log. Keep it open so that you can examine it carefully.

Examining the Execution Log

Notice that the first line in the execution log marks the `EXECUTION_STARTED` event and that the last line is the `EXECUTION_FINISHED` event. Everything in between is the execution context.

Let's take a closer look at what happens. The first `CODE_UNIT_STARTED` event marks when the code from the Execute Anonymous window was kicked off. This line is highlighted in red in the image below.



[Apex & .NET Basics \(https://trailhead.salesforce.com/content/learn/modules/apex_basics_dotnet?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](https://trailhead.salesforce.com/content/learn/modules/apex_basics_dotnet?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Understanding Execution Context

12:30:30:137	HEAP_ALLOCATE	[1]]Bytes:4
12:30:30:137	HEAP_ALLOCATE	[2]]Bytes:14
12:30:30:137	VARIABLE_ASSIGNMENT	[1]]this.Name Test Account 2" 0x2d84256d
12:30:30:137	HEAP_ALLOCATE	[3]]Bytes:13
12:30:30:137	VARIABLE_ASSIGNMENT	[1]]this.NumberOfEmployees 50 0x2d84256d
12:30:30:137	HEAP_ALLOCATE	[5]]Bytes:13
12:30:30:137	VARIABLE_ASSIGNMENT	[1]]this.BillingCity "San Francisco" 0x2d84256d
12:30:30:137	VARIABLE_ASSIGNMENT	[1]]acct {"Name":"Test Account 2",Phone":"(415)555-8989",NumberOfEmployees":50,"BillingCity":"San Francisco"} 0x2d84256d
12:30:30:138	STATEMENT_EXECUTE	[6]]
12:30:30:138	HEAP_ALLOCATE	[6]]Bytes:8
12:30:30:138	DML_BEGIN	[6]]Op:Insert Type:Account Rows:1
12:30:30:138	HEAP_ALLOCATE	[EXTERNAL]]Bytes:8
12:30:30:413	CODE_UNIT_STARTED	[EXTERNAL]]01qd00000000Kbd4 sarahasnoLimits.AccountTrigger on Account trigger event BeforeInsert for [new]
12:30:30:413	HEAP_ALLOCATE	[EXTERNAL]]Bytes:8
12:30:30:413	HEAP_ALLOCATE	[EXTERNAL]]Bytes:4
12:30:30:413	VARIABLE_SCOPE_BEGIN	[1]]this sarahasnoLimits.AccountTrigger true false
12:30:30:413	VARIABLE_ASSIGNMENT	[1]]this {} 0x5bf435da
12:30:30:413	HEAP_ALLOCATE	[EXTERNAL]]Bytes:4
12:30:30:413	VARIABLE_SCOPE_BEGIN	[1]]this sarahasnoLimits.AccountTrigger true false
12:30:30:413	VARIABLE_ASSIGNMENT	[1]]this {} 0x5bf435da
12:30:30:413	STATEMENT_EXECUTE	[1]]
12:30:30:413	STATEMENT_EXECUTE	[4]]
12:30:30:413	CUMULATIVE_LIMIT_USAGE	
12:30:30:413	LIMIT_USAGE_FOR_NS	(default)]
12:30:30:000	LIMIT_USAGE_FOR_NS	Number of SOQL queries: 0 out of 100
12:30:30:000	LIMIT_USAGE_FOR_NS	Number of query rows: 0 out of 50000
12:30:30:000	LIMIT_USAGE_FOR_NS	Number of SOSL queries: 0 out of 20
12:30:30:000	LIMIT_USAGE_FOR_NS	Number of DML statements: 0 out of 150
12:30:30:000	LIMIT_USAGE_FOR_NS	Number of DML rows: 0 out of 10000
12:30:30:000	LIMIT_USAGE_FOR_NS	Maximum CPU time: 0 out of 10000

The second CODE_UNIT_STARTED line that is highlighted represents when code for the BeforeInsert event was executed.

You can't see this in the image, but if you're following along with your own instance of Developer Console, scroll down further in the results and look for other instances of CODE_UNIT_STARTED. You should see at least one more instance representing when code for the AfterInsert event was executed. If you had created workflow rules that fired when a new account was created, they too would show up in the execution log. All this code operates under the same execution context, and thus, is subject to the same set of governor limits.

You may be asking yourself, why is this so important to understand? Well, because Salesforce is a multi-tenanted environment, those governor limits are vital in keeping each instance of a Salesforce org from consuming too many resources. Essentially, they keep the whole system from crashing.

Working with Limits

And this brings us back to the subject of working with limits. The two limits you will probably be the most concerned with involve the number of SOQL queries or DML statements. These tend to trip up developers new to the platform, so we wanted to spend some extra time focusing on how to avoid them.



There are lots of limits to be aware of, and they tend to change with each major release. Additionally, it's not uncommon for limits to get looser rather than tighter, so be sure to check out the latest by looking at the Execution Governors and Limits link in Resources.

Working in Bulk

Many developers fall into a common trap of designing their code to work with a single record. They quickly learn that on the Lightning Platform this can be a huge mistake.

Apex triggers can receive up to 200 records at once. Currently, the synchronous limit for the total number of SOQL queries is 100, and 150 for the total number of DML statements issued. So, if you have a trigger that performs a SOQL query or DML statement inside of a loop and that trigger was fired for a bulk operation, guess what?

SCREECH!!!



Maybe you noticed, maybe you didn't, but the trigger handler code we created earlier didn't use a bulk pattern, and Apex & .NET Basics (content/learn/modules/apex_basics_dotnet/trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Understand Execution Context

```
1 public with sharing class AccountHandler {
2     public static void CreateNewOpportunity(List<Account> accts) {
3         for(Account acct : accts) {
4             Opportunity opp = new Opportunity();
5             opp.Name = acct.Name + ' Opportunity';
6             opp.AccountId = acct.Id;
7             opp.StageName = 'Prospecting';
8             opp.CloseDate = System.Today().addMonths(1);
9             insert opp;
10        }
11    }
12 }
```



Notice that the insert DML operation, `insert opp` , is inside the for loop. This is bad, very bad, and something to always avoid.

Luckily for us, we can fix this code by changing it to write to a list variable inside of the loop and then insert the contents of the list in one step.

1. In Developer Console, select **File > Open**.
2. For the entity type, select **Classes**. Select **AccountHandler** as the entity.
3. Click **Open**.
4. Delete the existing code, and insert the following snippet:

```
1 public with sharing class AccountHandler {
2     public static void CreateNewOpportunity(List<Account> accts) {
3         List<Opportunity> opps = new List<Opportunity>();
4         for(Account acct : accts) {
5             Opportunity opp = new Opportunity();
6             opp.Name = acct.Name + ' Opportunity';
7             opp.AccountId = acct.Id;
8             opp.StageName = 'Prospecting';
9             opp.CloseDate = System.Today().addMonths(1);
10            opps.add(opp);
11        }
12        if(opps.size() > 0) {
13            insert opps;
14        }
15    }
16 }
```



5. Press **Ctrl + S** to save your class.

Now that we fixed the trigger handler code, let's test to make sure that the trigger can handle a load of 200 records. As we're sure you know, writing unit tests to ensure that the code works is a best practice.

1. In Developer Console, select **File > New > Apex Class**.
2. Enter `AccountTrigger_Test`  for the class name and click **OK**.
3. Delete the existing code, and insert the following snippet:



```

5      // Create 200 new Accounts
6      List<Account> accts = new List<Account>();
7      for(Integer i=0; i < 200; i++) {
Apex & .NET Basics (/content/learn/modules/apex_basics_dotnet?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)
8          Account acct = new Account(Name='Test Account ' + i);
9          accts.add(acct);
10
11         // Perform Test
12         Test.startTest();
13         insert accts;
14         Test.stopTest();
15         // Verify that 200 new Accounts were inserted
16         List<Account> verifyAccts = [SELECT Id FROM Account];
17         Assert.areEqual(200, verifyAccts.size());
18         // Also verify that 200 new Opportunities were inserted
19         List<Opportunity> verifyOpps = [SELECT Id FROM Opportunity];
20         Assert.areEqual(200, verifyOpps.size());

```

Understand Execution Context

Show More

4. Press **Ctrl + S** to save your class.
5. Select **Test > New Run**.
6. Select **AccountTrigger_Test** as the TestClass, and **TestCreateNewAccountInBulk** as the test method.
7. Click **Run**.
8. Select the **Tests** tab and verify that the test runs to completion with no failures, as indicated by a green checkmark in the Status column.



Although we haven't yet talked about unit tests, don't worry. They work much the same way in the Lightning Platform as they do in .NET, with just a few different keywords. You'll catch on pretty quickly as to how they're structured. Learn more about testing Apex triggers by checking out the link in Resources.

Tell Me More

Apex uses a familiar try-catch-finally block to handle exceptions. But your catch statement and possible rollback could differ depending on where the Apex code is executed. Check out the link in Resources about best practices when dealing with try, catch, and rollback in Apex.

There is no such thing as an application or session variable in the Lightning Platform. If you need data to persist between classes, you do have static variables, but keep in mind that static variables in the Lightning Platform don't work the same as they do in .NET. In the Lightning Platform world, a static variable can only persist information within a single execution context, although other options for maintaining data across trigger invocations are available. Check out the Advanced Apex links in Internet Resources to learn more.

When working with limits, there are many tradeoffs to consider, especially for developers of managed packages. By the way, Salesforce partners typically use managed packages to distribute and sell applications. In this module, we just skimmed the surface of what you need to know. If you're serious about Apex development, check out the Advanced Apex links in Internet Resources.



- [Trailhead Trail: Developer Beginner](https://trailhead.salesforce.com/content/learn/trails/force_com_dev_beginner)

(https://trailhead.salesforce.com/content/learn/trails/force_com_dev_beginner)

Apex & .NET Basics (https://trailhead.salesforce.com/content/learn/modules/apex_basics_dotnet?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

- [Apex Developer Guide: Executing Governors and Limits](https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_gov_limits.htm) (https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_gov_limits.htm)

- [Trailhead: Apex Testing](https://trailhead.salesforce.com/content/learn/modules/apex_testing) (https://trailhead.salesforce.com/content/learn/modules/apex_testing)

[Understand Execution Context](#)

GET READY

You'll be completing this unit in your own hands-on org. Click **Launch** to get started, or click the name of your org to choose a different one.

YOUR CHALLENGE

Write an Apex trigger that modifies Account fields before inserting records.

Write an Apex trigger that fires before records are inserted and ensures that the ShippingState field has the same value as the BillingState field.

- Create an Apex class that contains a public static method:
 - Name: **AccountTriggerHandler**
 - Method name: **CreateAccounts**
 - The method must accept a List of Account objects
 - For each Account record, before saving, update the the **ShippingState** field to have the same value as the **BillingState** field
- Create an Apex Trigger:
 - Name: **AccountTrigger**
 - sObject: **Account**
 - Fires: **before records are inserted**
 - Your trigger must call the **AccountTriggerHandler.CreateAccounts()** method with the collection of new records
 - Use the **isBefore** and **isInsert** trigger context variables
- Create a test class:
 - Name: **AccountTriggerTest**
 - Your test class must insert 200 Account records with a BillingState of **CA**
 - After the insert, test to ensure that all 200 records have a ShippingState of **CA**
- Before verifying this challenge, run your test class at least once using the Developer Console Run All feature

Choose hands-on org

Koushik nelluri Salesforce CRM

Created on 5/9/2026



Launch

Check Challenge to Earn 500 Points



Trails	Maintain Certifications	Events	Sales Enablement
Trailmixes	Verify Certifications	Agentblazers	Trailblazer Store
Apex & .NET Basics (/content/learn/modules/apex_basics_dotnet?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)		Salesforce Developers	Salesforce Help
Superbadges			
Trailhead Academy		Salesforce Admins	
Understand Execution Context		Code of Conduct	
Career Paths		Report Illegal Content (EU)	

Download Trailhead GO



© 2026 Salesforce, Inc. All rights reserved.

[Privacy Statement](#) [Terms of Use](#) [Use of Cookies](#) [Trust](#) [Accessibility](#) [Cookie Preferences](#) [Your Privacy Choices](#)

English